

Round Robin (RR)

Round Robin gives each ready process a turn on the CPU. An unfinished process goes to the back of the queue when its turn ends.

1. Service time and time slice

The visualizer uses one CPU, whole ticks, and CPU-only processes. Switching processes takes no time.

Service time is the total CPU work a process needs to finish. The **quantum**, also called the **time slice**, limits how many CPU ticks it can use in one turn. The default quantum is 2; any positive whole number is allowed.

For example, a process needing five service ticks with a quantum of 2 needs turns of two, two, and one tick. Waiting between turns does not reduce its remaining service. RR has no separate allotment or priority levels.

A ready process has arrived and is waiting for the CPU. A future process has not arrived yet. In [B, C], B is the queue head. The short examples under the rules are separate scenarios.

2. Scheduling rules

Rule 1. Add arrivals to the queue tail

Existing waiting processes stay ahead of new arrivals. If several processes arrive together, add them in input order. If arrival and quantum expiry coincide, admit the arrivals before requeueing the unfinished process.

Explanation. An unfinished process whose turn has ended returns behind both the existing queue and any arrivals at that boundary. The same-time ordering matters because RR selects the queue head.

Example. A's quantum ends at time 2 while B is waiting. C and D arrive at time 2, listed in that order. Admit C and D, then append A:

<i>Stage</i>	<i>Ready queue</i>
After admitting C and D	B, C, D
After requeueing A	B, C, D, A
After dispatching B	C, D, A

Rule 2. Give the queue head a fresh quantum

When the CPU is free, remove the first ready process and set its quantum usage to zero. Each tick it runs reduces remaining service by one and increases quantum usage by one.

Explanation. The quantum measures this turn's CPU use, not the total work left. An arrival joins the queue and cannot interrupt the turn. Waiting uses none of a process's quantum.

Example. B starts with five service ticks and a quantum of 2. After one tick, B has four service ticks left and has used 1/2 of its quantum. If C arrives now, B continues for its second tick. B's turn then ends with three service ticks left.

Rule 3. Complete before considering rotation

If the process has no service left, remove it. This applies even when its last service tick also uses the last tick of its quantum. A finished process is never sent back to the queue.

Explanation. Quantum expiry matters only for unfinished work. Checking completion first prevents a finished process from being requeued or receiving another turn.

Example. A needs two service ticks and starts at time 0 with quantum 2. At time 2, both its service and quantum end. Remove A. If B is already waiting and C arrives at time 2, the queue is [B, C] before dispatch; it contains no A.

Rule 4. Rotate an unfinished process when its quantum expires

Reset its quantum usage and append it to the queue tail. Preserve its remaining service. If other processes are waiting, they run first.

Explanation. A turn ending is different from a process finishing. Reset only the turn counter and keep the work still needed. If the process is alone, it may immediately start a fresh turn without an idle tick.

Example. A needs five service ticks and runs from 0 to 2 with quantum 2. It has three ticks left. If B is waiting, requeue A to form [B, A]; B runs next. If nobody is waiting, A immediately starts its next turn at time 2. Its remaining turns are 2 to 4 and 4 to 5.

Rule 5. Idle only when no process is ready

Wait for the next arrival if the CPU and ready queue are empty. Stop when all processes have finished.

Explanation. A future process is not yet available for a turn. An empty queue does not stop a process already running, and rotating a lone unfinished process does not make the CPU idle.

Example. A arrives at time 2 needing one tick; B arrives at time 5 needing one. With quantum 2, each finishes within its first turn. The CPU is idle before A arrives and again between their arrivals:

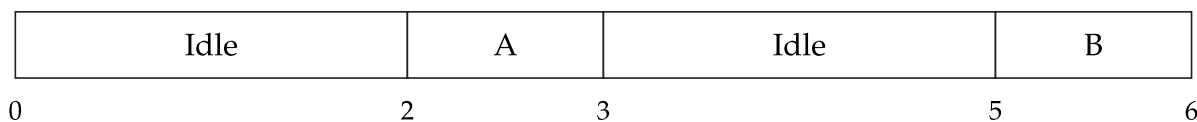


Figure 1. CPU schedule (time in ticks).

3. Events at a tick boundary

After accounting for the preceding tick:

1. Remove the running process if it has finished.
2. If an unfinished process has used its quantum, mark its turn as ended.
3. Add all arrivals at the current time.
4. Append the process whose turn ended, if any, with quantum usage reset.
5. If the CPU is free, dispatch the queue head.

If the running process has neither finished nor exhausted its quantum, it continues. Dispatching or requeueing takes no CPU tick.

Rule 1's example follows this order: existing B stays first, arrivals C and D enter next, and the unfinished A goes last. If A had completed instead, Rule 3 would remove it before those arrivals.

4. Worked example

Input order is A, B, C. Use quantum 2. All times are in ticks.

<i>Process</i>	<i>Arrival</i>	<i>Service</i>
A	0	5
B	1	2
C	1	1

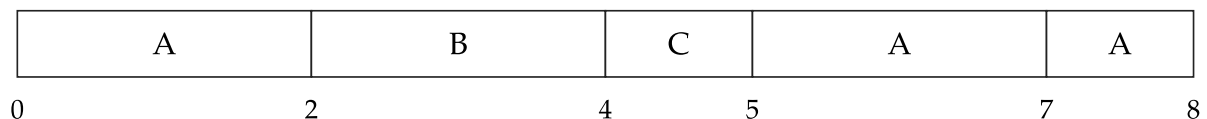


Figure 2. CPU schedule (time in ticks).

A's first turn ends at time 2, with three service ticks left. B then finishes exactly at its quantum boundary. C needs only one tick, so it releases the CPU at time 5 without using its full quantum. A runs from 5 to 7, then immediately takes another turn because nobody else is waiting. It finishes at time 8.

The boundary at time 7 separates two turns of A, although A uses the CPU on both sides. All three processes have finished at time 8.